

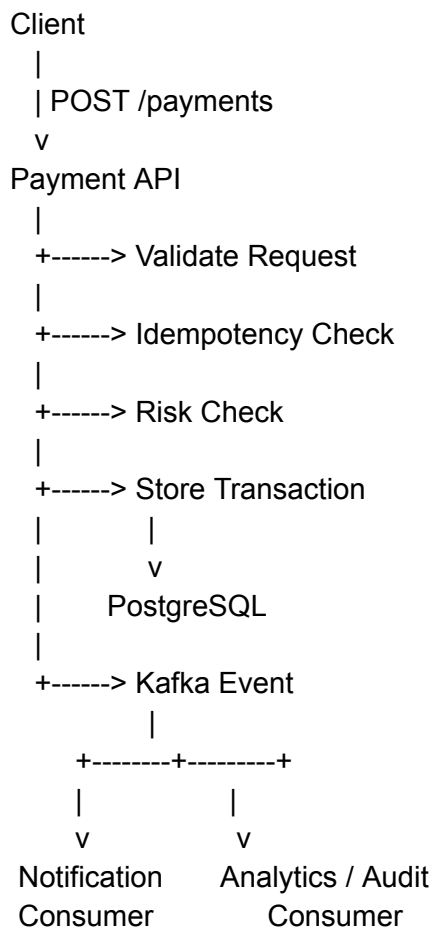
PayFlow — Event-Driven Payment Processing System

1. Project objective

Build a backend service that accepts digital payment requests, validates them, performs a simple risk assessment, stores the transaction safely, prevents accidental duplicate payments, and publishes payment events to Kafka so downstream services can react asynchronously.

The project should demonstrate that you understand how a real backend system is designed rather than just how to build CRUD APIs.

A simplified flow is:



This is particularly aligned with Navi because its internship description explicitly asks for familiarity with event-driven architectures and messaging systems such as Kafka/RabbitMQ, along with backend/API security and appropriate unit, contract and component testing.

2. Problem statement

Modern payment systems cannot assume that every request arrives exactly once or that every component is always available.

For example, suppose a user tries to make a ₹2,000 payment and their network temporarily fails. The application may retry the request.

Without safeguards, the backend could process:

Request 1 → ₹2,000 payment

Network timeout

Request 2 → ₹2,000 payment again

The user could accidentally be charged twice.

At the same time, several other systems may need to react after a successful payment:

Notification Service

Analytics Service

Audit Service

Fraud Monitoring

Rewards Service

Making the Payment API synchronously contact every one of these services would make the payment slower and more fragile.

Therefore, PayFlow should solve three primary problems:

Problem	Solution
Duplicate payment requests	Idempotency keys
Multiple downstream actions	Event-driven architecture using Kafka
Suspicious transactions	Simple rule-based risk engine

The project's core design principle is therefore:

Process the financial transaction reliably in the core service, and perform secondary actions asynchronously through events.

3. Scope

The system is intentionally not a complete UPI/payment gateway.

In scope

Create payment

Retrieve payment

Payment status tracking

Input validation

Risk assessment

Idempotency

PostgreSQL persistence

Kafka event publishing

Kafka consumer

Notifications/audit events
API authentication
Error handling
Unit/integration testing
Docker-based local deployment
Basic observability

Out of scope

Actual bank integration
Real UPI infrastructure
Real money movement
PCI-DSS implementation
Complex ML fraud detection
Distributed ledger
Settlement/reconciliation network
Mobile frontend
Real SMS/email delivery

That boundary is important in an interview. It shows that you understand **scope management**.

4. Main actors

There are four logical actors.

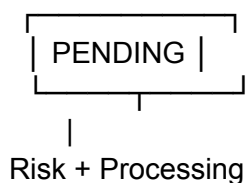
Actor	Role
Client	Creates and queries payments
Payment Service	Validates and processes payment requests
Kafka	Carries payment lifecycle events
Event Consumer	Reacts asynchronously to successful/failed payments

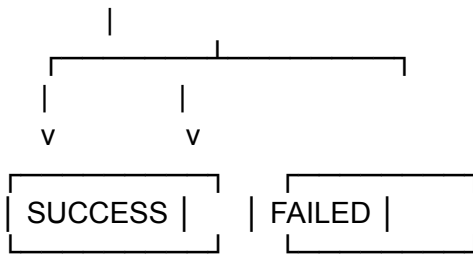
You don't need separate microservices for everything.

For a two-day project, one FastAPI application plus one Kafka consumer is enough.

5. Payment lifecycle

Keep the transaction lifecycle simple.





Possible statuses:

PENDING
SUCCESS
FAILED
REJECTED

REJECTED is optional, but it helps distinguish:

FAILED → technical/payment failure
REJECTED → risk engine refused payment

6. High-level functional requirements

I would actually document these as numbered requirements in the GitHub README. It makes the project look much more professionally engineered.

ID	Functional requirement	Priority
FR-01	The system shall allow a client to create a payment	Must
FR-02	The system shall validate sender, receiver and payment amount	Must
FR-03	The system shall generate a unique transaction ID for every new payment	Must
FR-04	The system shall persist payment information in PostgreSQL	Must
FR-05	The system shall maintain a payment status	Must
FR-06	The system shall prevent duplicate payment processing using an idempotency key	Must
FR-07	The system shall perform rule-based risk checks before approving a payment	Must
FR-08	The system shall publish payment events to Kafka	Must
FR-09	A consumer shall asynchronously process successful payment events	Must

FR-1 0	The system shall allow payment lookup using transaction ID	Must
FR-1 1	The system shall return appropriate HTTP errors for invalid requests	Must
FR-1 2	Protected endpoints shall require basic authentication/API credentials	Should
FR-1 3	The system shall record transaction timestamps and event history	Should
FR-1 4	The system shall provide health-check endpoints	Should
FR-1 5	The system shall expose basic operational metrics/logging	Could

Now let's define the important ones properly.

7. FR-01 — Create a payment

Endpoint:

POST /payments

Example request:

```
{
  "sender_id": 101,
  "receiver_id": 205,
  "amount": 1250.00,
  "currency": "INR"
}
```

Request header:

Idempotency-Key: 71eab261-928d-4ea7

Response:

```
{
  "transaction_id": "pay_82b94c",
  "sender_id": 101,
  "receiver_id": 205,
  "amount": 1250.00,
  "currency": "INR",
  "status": "SUCCESS"
}
```

Use HTTP:

201 Created

for a new payment.

8. FR-02 — Validate payment request

Before processing the transaction, validate:

amount > 0

sender_id exists

receiver_id exists

sender_id != receiver_id

currency == INR

Idempotency-Key exists

Examples:

amount = -500 → reject

sender == receiver → reject

missing sender → reject

missing key → reject

Return:

400 Bad Request

or:

422 Unprocessable Entity

FastAPI will naturally handle several schema-validation cases.

9. FR-03 — Unique transaction IDs

Every real payment should receive a unique identifier.

For example:

pay_f47ac10b

Internally you can simply use:

UUID

This ID becomes the identifier used across:

database

Kafka

logs
API responses

That gives you a useful distributed-systems concept to discuss:

correlation IDs / transaction IDs.

10. FR-04 — Persist transaction

Create a **payments** table.

Suggested schema:

payments

id
transaction_id
sender_id
receiver_id
amount
currency
status
risk_level
idempotency_key
created_at
updated_at

Example:

transaction_id : pay_f12345
sender_id : 101
receiver_id : 205
amount : 1250
currency : INR
status : SUCCESS
risk_level : LOW

You can enforce:

UNIQUE(idempotency_key)
UNIQUE(transaction_id)

This gives you protection at both application and database level.

11. FR-05 — Payment state management

Initially:

status = PENDING

After validation and risk checks:

LOW risk

PENDING → SUCCESS

or:

HIGH risk

PENDING → REJECTED

You don't need a complicated state machine library.

Just make the valid transitions explicit inside your service layer.

12. FR-06 — Idempotency

This should be one of your project's headline features.

Client sends:

Idempotency-Key: payment-request-xyz

First request:

POST /payments

Idempotency-Key: ABC123

→ transaction created

Second identical request:

POST /payments

Idempotency-Key: ABC123

→ existing transaction returned

It should NOT create another payment.

Conceptually:

```
existing = find_by_idempotency_key(key)
```

```
if existing:
```

```
    return existing
```

```
create_new_payment()
```

Also add a unique database constraint.

That gives you two levels of defence:

Application-level check

+

13. FR-07 — Risk engine

Don't build machine learning here.

A deterministic rule engine is much easier to implement and explain.

Example rules:

Amount > ₹50,000

↓

HIGH RISK

Another rule:

More than 5 transactions
from same user within 60 seconds

↓

HIGH RISK

You can calculate:

risk_score = 0

amount > 50000 +50
rapid transaction count +50

Then:

0–49 → LOW
50+ → HIGH

For the first implementation, even one or two rules is enough.

The important point is separating:

payment processing logic

from

risk evaluation logic

For example:

```
services/  
  payment_service.py  
  risk_service.py
```

That makes the architecture cleaner.

14. FR-08 — Publish Kafka event

After the payment reaches its final state, publish an event.

Topic:

payment-events

Successful event:

```
{
  "event_type": "PAYMENT_COMPLETED",
  "transaction_id": "pay_f12345",
  "sender_id": 101,
  "receiver_id": 205,
  "amount": 1250,
  "timestamp": "2026-09-15T12:00:00Z"
}
```

Rejected payment:

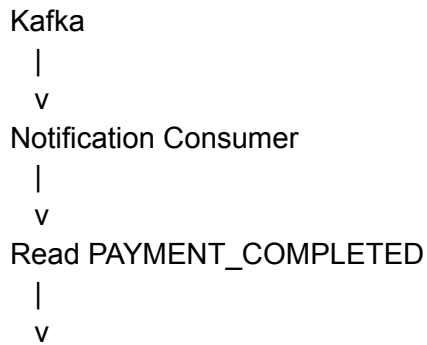
```
{
  "event_type": "PAYMENT_REJECTED",
  "transaction_id": "pay_f12345",
  "reason": "HIGH_RISK"
}
```

The event should contain enough information for downstream services to react without needing to know the internal structure of your Payment Service.

15. FR-09 — Kafka consumer

Your consumer can simulate a notification service.

Flow:



Log:
"₹1250 successfully sent from user 101 to user 205"

Optionally persist it in:

notifications

table.

Example:

notifications

id

transaction_id

message

created_at

That's enough to demonstrate asynchronous processing.

16. FR-10 — Get transaction

Endpoint:

GET /payments/{transaction_id}

Response:

```
{
  "transaction_id": "pay_f12345",
  "amount": 1250,
  "status": "SUCCESS",
  "risk_level": "LOW",
  "created_at": "..."
}
```

Unknown transaction:

404 Not Found

17. FR-11 — Error handling

Create consistent error responses.

For example:

```
{
  "error": "INVALID_PAYMENT_AMOUNT",
  "message": "Payment amount must be greater than zero"
}
```

Possible errors:

INVALID_AMOUNT
USER_NOT_FOUND
SAME_SENDER_RECEIVER
PAYMENT_NOT_FOUND
HIGH_RISK_TRANSACTION
UNAUTHORIZED
INTERNAL_ERROR

This ties nicely to your Uber experience because your current resume already describes implementing a dedicated frontend/backend error contract.

18. FR-12 — API security

Keep security deliberately basic.

You don't need OAuth infrastructure.

Implement either:

API Key authentication

or:

JWT authentication

For two days, I recommend:

JWT if you're comfortable with it; API key otherwise.

Also validate all inputs and avoid exposing database/internal errors.

Example:

Authorization: Bearer <token>

Don't spend half the project building user registration.

19. FR-13 — Audit information

Store:

created_at
updated_at
transaction status
risk classification

Also log important transitions:

PAYMENT_RECEIVED
PAYMENT_VALIDATED
RISK_CHECK_COMPLETE
PAYMENT_COMPLETED
EVENT_PUBLISHED

Then you can follow one transaction through the system using its transaction ID.

20. FR-14 — Health endpoint

Very easy but surprisingly professional.

GET /health

Response:

```
{
  "status": "healthy"
}
```

Better:

```
{
  "api": "healthy",
  "database": "healthy",
  "kafka": "healthy"
}
```

You can keep the first version simple.

21. Non-functional requirements

These matter because Navi specifically talks about performant and maintainable code, low-level design, customer experience, product performance and production-quality engineering. PS-II Sem II 26-27 at Navi Limited, Bengaluru.pdfPDF

ID	Non-functional requirement	Proposed target
NFR-01	Reliability	Duplicate requests must not create duplicate transactions
NFR-02	Performance	Typical API response under ~300 ms locally, excluding external Kafka delays
NFR-03	Scalability	Downstream processing should be decoupled through Kafka
NFR-04	Maintainability	Layered/modular service architecture
NFR-05	Testability	Business logic separated from HTTP/database code where practical
NFR-06	Security	Authentication, validation and safe error messages
NFR-07	Observability	Structured logs with transaction IDs

NFR-08	Availability	Failure of notification consumer should not prevent payment creation
NFR-09	Data integrity	Database constraints and controlled transaction states
NFR-10	Deployability	Entire system runnable using Docker Compose
NFR-11	Documentation	README with APIs, architecture and setup
NFR-12	Extensibility	Additional Kafka consumers should be addable without changing payment logic

22. NFR-01 — Reliability

This is arguably the most important requirement.

The system should guarantee:

One logical request



At most one payment transaction

even when the client retries.

That's where idempotency comes in.

You should explicitly test it.

23. NFR-02 — Performance

Don't promise enterprise-scale throughput.

Simply measure your own application.

For example, later you can say:

Average response time: X ms

p95 response time: Y ms

Run perhaps:

100
500
1000

local requests using a simple load testing tool.

If you don't have time, performance testing is optional.

But having even one measured benchmark makes the project more credible.

24. NFR-03 — Scalability

Your architecture supports scalability because downstream services don't need to execute inside the Payment API request.

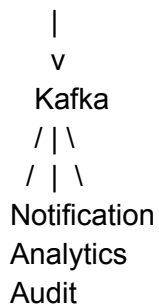
Instead of:

Payment API

- notification
- analytics
- rewards
- audit

you have:

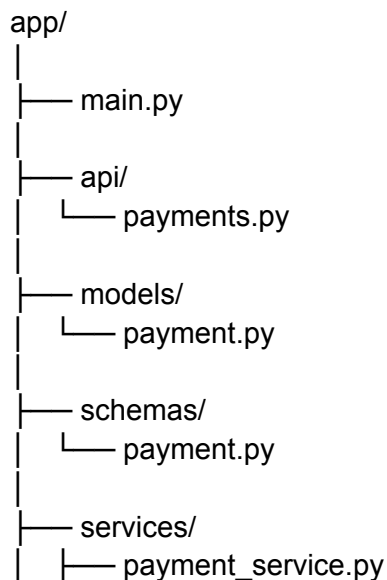
Payment API

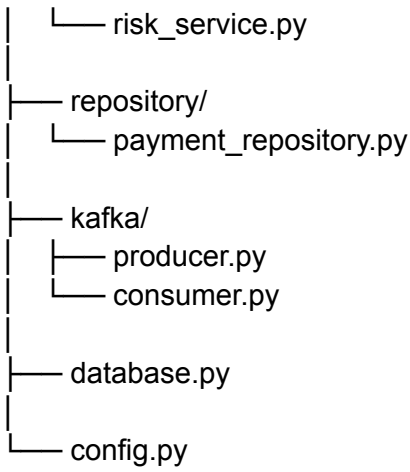


That is one of the primary advantages of event-driven architecture.

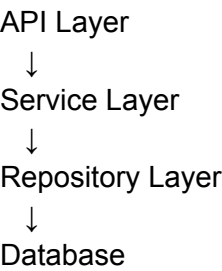
25. NFR-04 — Maintainability

Use a basic layered structure:





The logic should roughly follow:



Kafka sits alongside this as infrastructure.

26. NFR-05 — Testing

Have three broad layers.

Unit tests

Test:

- risk engine
- payment validation
- idempotency logic
- state transition

without Kafka/PostgreSQL where possible.

API tests

Test:

- POST /payments
- GET /payments/{id}
- invalid input
- authentication

Integration tests

Test at least:

API → PostgreSQL

and ideally:

Producer → Kafka → Consumer

You don't need a huge test suite.

Around **8–12 sensible tests** is enough for this project.

27. NFR-06 — Security

At a high level:

Authenticate requests

Validate inputs

Do not expose stack traces

Use parameterised ORM queries

Do not hard-code credentials

Store secrets in environment variables

`.env`:

```
DATABASE_URL=
```

```
KAFKA_BOOTSTRAP_SERVER=
```

```
JWT_SECRET=
```

Never commit `.env`.

Commit:

```
.env.example
```

instead.

28. NFR-07 — Observability

Every log should ideally contain:

transaction_id

status

timestamp

Example:

```
INFO transaction_id=pay_123 payment_received
```

```
INFO transaction_id=pay_123 risk_check=LOW
```

```
INFO transaction_id=pay_123 status=SUCCESS
```

INFO transaction_id=pay_123 kafka_event_published

That makes debugging much easier.

You already have genuine observability experience from Uber, where you worked with job/workflow metrics, error analysis and Grafana, so this is particularly natural to connect to your resume.

29. Recommended database model

For the two-day version, you really only need one important table.

PAYMENTS

id	UUID
transaction_id	VARCHAR UNIQUE
idempotency_key	VARCHAR UNIQUE
sender_id	INTEGER
receiver_id	INTEGER
amount	DECIMAL
currency	VARCHAR
status	VARCHAR
risk_level	VARCHAR
created_at	TIMESTAMP
updated_at	TIMESTAMP

Optionally:

NOTIFICATIONS

id
transaction_id
message
created_at

Don't build an elaborate Users schema unless you finish early.

30. API specification

The whole API can realistically be only four endpoints:

Method	Endpoint	Purpose
POST	/payments	Create payment
GET	/payments/{transaction_id}	Fetch payment
GET	/payments	List transactions

That's enough.

You don't need 15 endpoints for the project to look impressive.

31. Kafka design

Use one topic:

payment-events

Event types:

PAYMENT_COMPLETED

PAYMENT_REJECTED

PAYMENT_FAILED

Later, if asked how it could scale, you can explain that each type could move to separate topics.

But don't over-engineer your actual implementation.

32. End-to-end transaction sequence

This is the core flow you should understand extremely well:

1. Client sends POST /payments
2. API authenticates request
3. Request schema is validated
4. Service checks idempotency key
5. Existing transaction?
 YES → return existing transaction
6. Run risk engine
7. Create transaction as PENDING
8. Risk acceptable?
 NO → mark REJECTED
 YES → mark SUCCESS
9. Commit transaction to PostgreSQL
10. Publish payment event to Kafka

11. Return transaction response to client

12. Kafka consumer receives event

13. Consumer records/logs notification

If you can explain those thirteen steps clearly, you understand the project.

33. Suggested two-day implementation order

Don't build things randomly. Build one complete vertical slice first.

Time	Goal
Day 1 morning	FastAPI + PostgreSQL + payment model
Day 1 afternoon	POST/GET APIs + validation
Day 1 evening	Idempotency + risk engine
Day 2 morning	Kafka producer + consumer
Day 2 afternoon	pytest suite + error handling
Day 2 evening	Docker Compose + README + architecture diagram

If time becomes tight, prioritise:

API
↓
PostgreSQL
↓
Idempotency
↓
Kafka
↓
Testing
↓
Risk engine
↓
Docker

The first five matter more than polishing extra features.

34. What not to add in these two days

Don't add Redis, Kubernetes, Elasticsearch, Prometheus, Grafana, multiple databases, ML fraud detection, five microservices and ten Kafka topics just to create an impressive technology list.

Your current resume already has significant systems exposure through Uber — Spark-on-Kubernetes, large-scale job processing, observability, cost reporting and deployment. RajRahul_Resume.pdfPDF

This project only needs to prove:

“I understand backend system design and can implement a clean event-driven service myself.”

That is much more convincing than a giant architecture where you cannot explain half the components.

35. Final project definition

I'd frame it on GitHub as:

PayFlow — Event-Driven Payment Processing Platform

PayFlow is a production-inspired backend payment processing system built using FastAPI, PostgreSQL and Kafka. It provides reliable transaction processing through idempotent APIs, rule-based risk assessment, persistent transaction state and asynchronous event processing. The project demonstrates event-driven architecture, API design, reliability, testing and backend engineering principles commonly used in financial systems.

That is the scope I would stick to.

Once you've built this exact version, the project will be **small enough to genuinely understand in two days but sophisticated enough to justify replacing StateWise on your Navi resume.**